GH-500

Enterprise GitHub Advanced Security End-to-End Hands-on Workshop

Beginner to Advanced • SecureBank enterprise scenario • 10 progressive stages

Workshop purpose
Build a single evolving application repository and use it to demonstrate dependency security, secret protection, CodeQL, custom queries, SARIF, SBOM, GitHub REST APIs, GitHub Actions and enterprise security governance.

Lab application: SecureBank – a fictional digital banking service used only for security training.

Instructor workbook • PowerShell-first commands • Complete file listings • Demonstrations and capstone

Important: the application intentionally contains vulnerable training code. It must remain isolated and must never be deployed to production.

How to use this workbook

This workbook is designed as a progressive lab. Start with a minimal Python application, then add security controls stage by stage. Keep the same repository throughout so participants can see how enterprise security controls interact.

Workshop outcomes

• Create and operate a GitHub repository for a realistic application scenario.

• Enable dependency visibility, Dependabot alerts and dependency review.

• Understand secret scanning, push protection and credential response.

• Run CodeQL analysis and write a simple organization-specific CodeQL query.

• Generate and upload SARIF from a third-party scanner.

• Generate SPDX and CycloneDX SBOMs and understand release evidence.

• Use Postman and GitHub REST APIs to retrieve security findings.

• Build CI workflows, pull request checks and security gates.

• Explain enterprise security configurations, rulesets, ownership and remediation campaigns.

Ten-stage roadmap

Stage	Focus	Level
1	Repository, application and security baseline	Beginner
2	Dependabot and supply chain security	Beginner–Intermediate
3	Secret Protection	Intermediate
4	CodeQL fundamentals	Advanced
5	Custom CodeQL query engineering	Expert
6	Third-party scanning and SARIF	Advanced
7	SBOM and supply chain governance	Advanced
8	REST API and security automation	Advanced
9	GitHub Actions and secure CI/CD	Advanced–Expert
10	Enterprise governance and capstone	Expert

Prerequisites

• Git, PowerShell and Python 3.12.

• GitHub account and a repository; an organization is preferred for enterprise administration labs.

• Visual Studio Code and Postman.

• Syft for local SBOM generation.

• CodeQL CLI is optional; use a supported environment and validate platform compatibility before the session.

• Access to GitHub security features appropriate to the repository visibility and plan.

Licensing and feature availability: Code Security, Secret Protection, security configurations and some alert capabilities vary by repository visibility, organization plan and purchased entitlements. Confirm access before delivery. If a feature is unavailable, use the relevant documentation walkthrough and keep the hands-on exercise within the licensed lab environment.

Enterprise scenario: SecureBank

SecureBank is a fictional digital banking service. It exposes customer account information, accepts transaction requests and contains a calculation endpoint. The workshop begins with intentionally unsafe code and outdated dependencies so that security tools have meaningful findings to discover.

Enterprise challenge

• Developers may introduce vulnerable dependencies.

• Credentials can be accidentally committed.

• Application code may contain exploitable patterns.

• The security team needs an inventory of software components.

• Findings from multiple scanners must be consolidated and triaged.

• Releases must be protected by enforceable security policies.

• Central teams need repeatable reporting across many repositories.

Target control architecture

Control layer	Capability	Workshop evidence
Source control	GitHub repository, pull requests, CODEOWNERS	Protected main branch and review
Supply chain	Dependency Graph, Dependabot, dependency review	Alerts and remediation PR
Secrets	Secret scanning and push protection	Blocked synthetic test push / alert triage
Code security	CodeQL and custom queries	Code scanning alerts
External scanning	Semgrep and SARIF	SARIF upload and findings
Inventory	SPDX / CycloneDX SBOM	Versioned component inventory
Automation	REST API and GitHub Actions	JSON security report and CI checks
Governance	Rulesets, security configurations, campaigns	Enterprise policy and capstone

Safety: Do not use real credentials, customer data, production banking code or production secrets in this lab. Use a dedicated test repository and synthetic data only.

Stage 1 – Enterprise repository and security baseline

Objective: create the SecureBank repository, run the application and establish the initial security policy.

1.1 Create and clone the repository

• Create a repository named securebank-gh500.

• Use a public repository only if organizational policy permits; otherwise use a private repository with required security entitlements.

• Initialize with a README and clone it locally.

PowerShell – clone and configure

```
git clone https://github.com/YOUR-USERNAME/securebank-gh500.git
cd securebank-gh500
code .
git config user.name "GH500 Workshop"
git config user.email "workshop@example.com"
```

1.2 Create app.py

Complete file: app.py

```
from flask import Flask, jsonify, request

app = Flask(__name__)

ACCOUNTS = {
    "1001": {"customer": "Customer A", "balance": 50000},
    "1002": {"customer": "Customer B", "balance": 75000}
}

@app.get("/")
def home():
    return jsonify({
        "application": "SecureBank",
        "version": "1.0",
        "status": "running"
    })

@app.get("/health")
def health():
    return jsonify({"status": "healthy"})

@app.get("/accounts/<account_id>")
def get_account(account_id):
    account = ACCOUNTS.get(account_id)
    if not account:
        return jsonify({"error": "Account not found"}), 404
    return jsonify(account)

@app.post("/transactions")
def create_transaction():
    data = request.get_json(silent=True) or {}
    account_id = data.get("account_id")
    amount = data.get("amount", 0)

    if account_id not in ACCOUNTS:
        return jsonify({"error": "Invalid account"}), 400

    if not isinstance(amount, (int, float)) or amount <= 0:
        return jsonify({"error": "Invalid transaction amount"}), 400

    if amount > ACCOUNTS[account_id]["balance"]:
        return jsonify({"error": "Insufficient balance"}), 400

    ACCOUNTS[account_id]["balance"] -= amount
    return jsonify({
        "status": "success",
        "account_id": account_id,
        "transaction_amount": amount
    })

@app.get("/calculate")
def calculate():
    expression = request.args.get("expression", "1+1")
    # INTENTIONALLY VULNERABLE TRAINING CODE
    result = eval(expression)
    return jsonify({"expression": expression, "result": result})

if __name__ == "__main__":
```

```
    app.run(host="127.0.0.1", port=5000, debug=False)
```

1.3 Create requirements.txt and requirements-dev.txt

Complete file: requirements.txt

```
Flask==2.2.2
Werkzeug==2.2.2
```

Complete file: requirements-dev.txt

```
-r requirements.txt
pytest==8.3.5
```

1.4 Create tests/test_app.py

Complete file: tests/test_app.py

```
import pytest
from app import app, ACCOUNTS

@pytest.fixture
def client():
    app.config["TESTING"] = True
    with app.test_client() as client:
        yield client

def test_home(client):
    response = client.get("/")
    assert response.status_code == 200
    assert response.json["application"] == "SecureBank"

def test_health(client):
    response = client.get("/health")
    assert response.status_code == 200
    assert response.json["status"] == "healthy"

def test_account_exists(client):
    response = client.get("/accounts/1001")
    assert response.status_code == 200
    assert response.json["customer"] == "Customer A"

def test_account_missing(client):
    response = client.get("/accounts/9999")
    assert response.status_code == 404

def test_valid_transaction(client):
    original_balance = ACCOUNTS["1001"]["balance"]
    response = client.post("/transactions", json={
        "account_id": "1001", "amount": 100
    })
    assert response.status_code == 200
    assert response.json["status"] == "success"
    ACCOUNTS["1001"]["balance"] = original_balance

def test_invalid_transaction(client):
    response = client.post("/transactions", json={
        "account_id": "1001", "amount": -100
    })
    assert response.status_code == 400
```

1.5 Create SECURITY.md

Complete file: SECURITY.md

```
# Security Policy

## Supported Versions

| Version | Supported |
|---|---|
| 1.x | Yes |

## Reporting a Vulnerability

Do not disclose vulnerabilities through public GitHub issues.
Use the organization's approved private security reporting process.

Include the affected component, vulnerability description,
reproduction steps, potential impact and suggested remediation.

## Response Process

1. Security team acknowledges the report.
2. Vulnerability is triaged.
3. Severity and business impact are assessed.
4. Remediation owner is assigned.
```

```
5. A fix is implemented and tested.
6. Security team validates closure.

## Security Principles

- Never commit production credentials.
- Use least-privilege permissions.
- Review dependencies before merging.
- Require security checks before production releases.
```

1.6 Install, run and test

PowerShell – local execution

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -r requirements-dev.txt
python app.py

# In another PowerShell window:
Invoke-RestMethod http://127.0.0.1:5000/
Invoke-RestMethod http://127.0.0.1:5000/health
Invoke-RestMethod http://127.0.0.1:5000/accounts/1001
python -m pytest tests -v
```

1.7 Commit baseline

PowerShell – commit

```
git add .
git commit -m "Initial SecureBank application"
git push origin main
```

Demonstration: show the application running, six tests passing, the unsafe eval() call in source and the initial repository security policy. Ask: does passing functional testing prove that the application is secure? No—functional correctness and security analysis are different objectives.

Stage 2 – Dependabot and supply chain security

Objective: understand dependency inventory, vulnerability alerts, automated updates and pull request dependency review.

2.1 Enable GitHub features

• Open repository Settings → Advanced Security.

• Enable Dependency Graph where available.

• Enable Dependabot alerts.

• Enable Dependabot security updates.

• Confirm repository plan and visibility support the required features.

2.2 Configure Dependabot

Complete file: .github/dependabot.yml

```
version: 2

updates:
  - package-ecosystem: "pip"
    directory: "/"
    schedule:
      interval: "weekly"
      day: "monday"
    open-pull-requests-limit: 10
    groups:
      python-minor-and-patch:
        update-types:
          - "minor"
          - "patch"

  - package-ecosystem: "github-actions"
    directory: "/"
    schedule:
      interval: "weekly"
    open-pull-requests-limit: 5
```

2.3 Configure dependency review

Complete file: .github/workflows/dependency-review.yml

```
name: Dependency Review

on:
  pull_request:
    branches:
      - main

permissions:
  contents: read
  pull-requests: write

jobs:
  dependency-review:
    name: Review Dependencies
    runs-on: ubuntu-latest
    steps:
      - name: Checkout
        uses: actions/checkout@v4

      - name: Dependency Review
        uses: actions/dependency-review-action@v4
        with:
          fail-on-severity: high
          comment-summary-in-pr: always
```

2.4 Demonstrate Dependabot

• Open Security → Dependabot and inspect an alert.

• Review affected package, installed version, fixed version, CVE, severity and dependency path.

• Open the proposed security update pull request.

• Review the version change, run tests and merge only after validation.

• Create a controlled PR that introduces a known vulnerable dependency and observe dependency review behavior.

2.5 Understand the feature boundaries

Feature	Purpose
Dependency Graph	Inventory of direct and transitive dependencies.
Dependabot Alerts	Detect known vulnerable dependencies.
Security Updates	Propose fixes for vulnerable dependencies.
Version Updates	Keep dependencies current.
Dependency Review	Assess dependency changes introduced by a PR.
SBOM	Exportable inventory of software components.

Discussion: a dependency alert is not the same as a dependency update. Review the proposed version change, compatibility, tests and release impact before merging.

Stage 3 – Secret Protection

Objective: demonstrate secret scanning, push protection, alert triage, custom patterns and credential response.

3.1 Enable secret scanning

• Open Settings → Advanced Security / Secret Protection.

• Enable secret scanning and push protection where entitled.

• Review supported secret types and alert notification configuration.

• Explore delegated bypass controls where supported.

3.2 Synthetic secret exercise

Create a temporary file named secret-lab.txt containing only this fake token-like value. It is not a real credential:

Synthetic demonstration string

```
ghp_1234567890123456789012345678901234567890
```

PowerShell – attempt a controlled push

```
git add secret-lab.txt
git commit -m "Demonstrate secret protection"
git push origin main
```

If GitHub recognizes the pattern, push protection may block the push. Detection depends on the enabled detector and current pattern behavior. If not recognized, use GitHub's documented test-secret examples for the specific detector; never substitute a real credential.

3.3 Create CODEOWNERS

Complete file: CODEOWNERS (replace placeholders)

```
# Security-sensitive files require security team review.
.github/ @YOUR-ORG/security-team
*.yml @YOUR-ORG/security-team
*.yaml @YOUR-ORG/security-team
requirements.txt @YOUR-ORG/security-team
requirements-dev.txt @YOUR-ORG/security-team
security-queries/ @YOUR-ORG/security-team
SECURITY.md @YOUR-ORG/security-team
```

3.4 Secret alert lifecycle

State / action	Meaning
Open	A secret exposure has been detected.
Resolved	The exposure has been addressed.
Revoked	The credential has been invalidated.
False positive	The detection is incorrect and has been reviewed.
Test credential	A non-production test value, with supporting evidence.

Enterprise rule: deleting a file is not sufficient. Treat a real exposed credential as compromised until it has been investigated, revoked or rotated, and any unauthorized use has been assessed.

3.5 Custom pattern exercise

Define a fictional SecureBank transaction-service credential pattern. Specify a pattern name, format, validation rules, positive test strings, negative test strings and false-positive exclusions. Use synthetic examples only.

Stage 4 – CodeQL fundamentals

Objective: understand CodeQL extraction, databases, query execution, query suites and GitHub code scanning.

4.1 CodeQL processing model

Step	What happens
Source	Application source code is checked out.
Extractor	Language-specific extraction captures code structure.
Database	CodeQL represents the source program in a queryable database.
Queries	Standard or custom QL queries analyze the database.
Results	Findings are emitted in SARIF and uploaded to GitHub.

Introduce these terms: extractor, database, QL language, query pack, query suite, source, sink, dataflow, taint tracking, security-extended queries, SARIF and alert lifecycle.

4.2 Create CodeQL workflow

Complete file: .github/workflows/codeql.yml – initial version

```
name: CodeQL Security Analysis

on:
  push:
    branches:
      - main
  pull_request:
    branches:
      - main
  schedule:
    - cron: "30 2 * * 1"
  workflow_dispatch:

permissions:
  contents: read
  security-events: write
  actions: read

jobs:
  analyze:
    name: CodeQL Analysis
    runs-on: ubuntu-latest
    timeout-minutes: 30
    permissions:
      contents: read
      security-events: write
      actions: read
    steps:
      - name: Checkout
        uses: actions/checkout@v4

      - name: Initialize CodeQL
        uses: github/codeql-action/init@v4
        with:
          languages: python
          queries: security-extended

      - name: Perform CodeQL Analysis
        uses: github/codeql-action/analyze@v4
        with:
          category: "/language:python"
```

4.3 Run and inspect

PowerShell – commit workflow

```
git add .
git commit -m "Add CodeQL security analysis"
git push origin main
```

• Open Actions → CodeQL Security Analysis.

• Inspect initialization, language detection, database creation and query execution.

• Review SARIF upload in the workflow logs.

• Open Security → Code scanning and inspect alerts.

4.4 Optional local CodeQL CLI

PowerShell – local analysis example

```
codeql version

codeql database create codeql-db `
  --language=python `
  --source-root=.

codeql database analyze codeql-db `
  security-queries/avoid-eval.ql `
  --format=sarif-latest `
  --output=codeql-results.sarif

Get-Content .\codeql-results.sarif -Raw
```

Validate CodeQL CLI availability, licensing terms and operating-system/architecture support before making local execution mandatory. GitHub-hosted Actions is the primary reproducible path in this workbook. For enterprise workflows, pin actions to approved immutable commit SHAs.

Stage 5 – Custom CodeQL query engineering

Objective: create a simple organization-specific query that detects Python eval() calls, then add it to the GitHub CodeQL workflow.

5.1 Create the custom query

Complete file: security-queries/avoid-eval.ql

```
/**
 * @name Avoid dynamic evaluation
 * @description Detects calls to Python eval().
 * @kind problem
 * @problem.severity error
 * @security-severity 8.0
 * @precision high
 * @id securebank/avoid-eval
 * @tags security
 *       external/cwe/cwe-95
 */

import python

from Call call, Name name

where
    call.getFunc() = name and
    name.getId() = "eval"

select call,
    "Avoid eval() with untrusted or dynamically supplied input. Use a safe parser or explicit validation."
```

5.2 Updated complete CodeQL workflow

Complete file: .github/workflows/codeql.yml – custom query version

```
name: CodeQL Security Analysis

on:
  push:
    branches:
      - main
  pull_request:
    branches:
      - main
  schedule:
    - cron: "30 2 * * 1"
  workflow_dispatch:

permissions:
  contents: read
  security-events: write
  actions: read

jobs:
  analyze:
    name: CodeQL Analysis
    runs-on: ubuntu-latest
    timeout-minutes: 30
    permissions:
      contents: read
      security-events: write
      actions: read
    steps:
      - name: Checkout
        uses: actions/checkout@v4

      - name: Initialize CodeQL
        uses: github/codeql-action/init@v4
        with:
          languages: python
          queries: |
            security-extended
            ./security-queries/avoid-eval.ql

      - name: Analyze
        uses: github/codeql-action/analyze@v4
        with:
          category: "/language:python"
```

5.3 Demonstrate and remediate

• Commit the query and updated workflow.

• Run CodeQL and inspect the custom alert for the calculate() function.

• Review source location and remediation guidance.

• Assign the finding, create a remediation PR and validate the fix.

5.4 Advanced query engineering topics

Topic	Advanced exercise
Metadata	Severity, precision, CWE and rule identifier.
AST queries	Identify unsafe syntax and API calls.
Dataflow	Follow values through program operations.
Taint tracking	Trace untrusted input to a dangerous sink.
Query packs	Package and version organization rules.
Query testing	Create expected positive and negative test cases.
Performance	Avoid unnecessarily expensive database scans.
Documentation	Provide developer-facing query guidance.

The example is a basic syntactic query, not a complete taint-tracking analysis. The advanced exercise should evolve it to trace HTTP request input to a dangerous execution sink, with query tests and reviewed query-pack metadata.

Stage 6 – Third-party scanning and SARIF

Objective: run an external scanner, generate SARIF and upload findings to GitHub Code Scanning.

6.1 SARIF architecture

SARIF (Static Analysis Results Interchange Format) provides a common format for static analysis results. It can connect findings from different scanners to GitHub Code Scanning, subject to supported SARIF schema and upload requirements.

6.2 Create Semgrep workflow

Complete file: .github/workflows/security.yml

```
name: Third Party Security Scanning

on:
  push:
    branches:
      - main
  pull_request:
    branches:
      - main
  workflow_dispatch:

permissions:
  contents: read
  security-events: write

jobs:
  semgrep:
    name: Semgrep Security Scan
    runs-on: ubuntu-latest
    steps:
      - name: Checkout
        uses: actions/checkout@v4

      - name: Setup Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.12"

      - name: Install Semgrep
        run: |
          python -m pip install semgrep

      - name: Generate SARIF
        run: |
          semgrep scan \
            --config auto \
            --sarif \
            --output semgrep-results.sarif

      - name: Upload SARIF
        if: always() && hashFiles('semgrep-results.sarif') != ''
        uses: github/codeql-action/upload-sarif@v3
        with:
          sarif_file: semgrep-results.sarif
          category: semgrep
```

6.3 Run and inspect

PowerShell – commit

```
git add .
git commit -m "Add Semgrep SARIF integration"
git push origin main
```

• Inspect the workflow run and Semgrep output.

• Review the SARIF file as an artifact or download it from the runner.

• Inspect rules, results, severity level, message and physical location.

• Open Code Scanning and review the uploaded findings.

• Compare Semgrep and CodeQL findings and decide which findings should block a release.

6.4 SARIF elements

Element	Purpose
version	SARIF format version.
runs	One or more tool execution records.
tool / driver	Scanner identity and metadata.
rules	Rule definitions.
results	Individual findings.
ruleId / level	Rule identifier and severity level.
locations	Source file and region.
message	Finding description.

Enterprise lesson: multiple scanners may detect the same underlying weakness. Define scanner ownership, categories, deduplication and triage procedures to reduce noisy reporting.

Stage 7 – SBOM and supply chain governance

Objective: generate SPDX and CycloneDX software bills of materials and understand release-level component inventory.

7.1 SBOM concepts

• Direct and transitive dependencies.

• Package versions and identifiers.

• Licenses and component relationships.

• Vulnerability intelligence and provenance.

• SPDX and CycloneDX formats.

• Release-specific SBOM retention.

7.2 Install Syft

PowerShell – verify installation

```
syft version
```

Install Syft using its official Windows installation method. Validate the executable and version before the workshop.

7.3 Create SBOM script

Complete file: scripts/generate-sbom.ps1

```
$ErrorActionPreference = "Stop"

$Root = Split-Path -Parent $PSScriptRoot
$OutputDirectory = Join-Path $Root "artifacts"

New-Item -ItemType Directory -Path $OutputDirectory -Force |
    Out-Null

$Requirements = Join-Path $Root "requirements.txt"

if (-not (Test-Path $Requirements)) {
    throw "requirements.txt was not found."
}

$SpdxOutput = Join-Path $OutputDirectory "securebank-spdx.json"
$CycloneDxOutput = Join-Path $OutputDirectory "securebank-cyclonedx.json"

Write-Host "Generating SPDX SBOM..."
syft scan "dir:$Root" --source-name securebank `
    -o "spdx-json=$SpdxOutput"

Write-Host "Generating CycloneDX SBOM..."
syft scan "dir:$Root" --source-name securebank `
    -o "cyclonedx-json=$CycloneDxOutput"

if (-not (Test-Path $SpdxOutput)) {
    throw "SPDX SBOM generation failed."
}

if (-not (Test-Path $CycloneDxOutput)) {
    throw "CycloneDX SBOM generation failed."
}

Write-Host ""
Write-Host "SBOM generation completed."
Write-Host "SPDX: $SpdxOutput"
Write-Host "CycloneDX: $CycloneDxOutput"
```

7.4 Run and inspect

PowerShell – generate and inspect SPDX

```
.\scripts\generate-sbom.ps1

Get-ChildItem .\artifacts

$sbom = Get-Content .\artifacts\securebank-spdx.json -Raw |
    ConvertFrom-Json

$sbom.packages | Select-Object name, versionInfo
```

7.5 GitHub SBOM and release evidence

Review the repository dependency graph and its SBOM export capability, if available. Explain the difference between GitHub's dependency graph, GitHub SBOM export, a locally generated SBOM and an SBOM attached to a release. Retain a release-specific SBOM as controlled evidence.

7.6 Vulnerability response exercise

Scenario: a critical vulnerability is published in a third-party package. Participants identify whether SecureBank is affected, which dependency introduced the package, which release contains the affected version, whether a fixed version exists, which remediation PR is required and which releases need investigation.

Stage 8 – GitHub REST API and security automation

Objective: retrieve repository security information using Postman and automate a consolidated report with PowerShell.

8.1 Token and Postman environment

For a lab, create a fine-grained personal access token restricted to the SecureBank repository. Grant only the read permissions required by the endpoints: metadata, Dependabot alerts, code scanning alerts and secret scanning alerts where available. For production automation, prefer a GitHub App with narrowly scoped permissions and short-lived installation tokens.

Variable	Value
baseUrl	https://api.github.com
owner	Your username or organization
repo	securebank-gh500
token	Fine-grained token; store as a secret

8.2 Collection-level headers

HTTP headers

```
Accept: application/vnd.github+json
Authorization: Bearer {{token}}
X-GitHub-Api-Version: 2022-11-28
```

8.3 API exercise matrix

Exercise	Method	Endpoint
Repository details	GET	/repos/{owner}/{repo}
Dependabot alerts	GET	/repos/{owner}/{repo}/dependabot/alerts
Dependabot alert	GET	/repos/{owner}/{repo}/dependabot/alerts/{alert_number}
Code scanning alerts	GET	/repos/{owner}/{repo}/code-scanning/alerts
Code scanning alert	GET	/repos/{owner}/{repo}/code-scanning/alerts/{alert_number}
Secret scanning alerts	GET	/repos/{owner}/{repo}/secret-scanning/alerts
Dependency graph SBOM	GET	/repos/{owner}/{repo}/dependency-graph/sbom
List workflows	GET	/repos/{owner}/{repo}/actions/workflows
Workflow runs	GET	/repos/{owner}/{repo}/actions/runs

For each endpoint demonstrate request construction, authentication, required permissions, HTTP status, JSON response, pagination and error handling. A 403 or 404 may indicate insufficient permissions, feature unavailability or resource visibility—not necessarily zero alerts.

8.4 Create PowerShell security report

Complete file: scripts/security-report.ps1

```
param(
    [Parameter(Mandatory = $true)]
    [string]$Owner,

    [Parameter(Mandatory = $true)]
    [string]$Repository
)

$ErrorActionPreference = "Stop"

if (-not $env:GITHUB_TOKEN) {
    throw "Set GITHUB_TOKEN before running this script."
}
```

```
$Headers = @{
    Authorization = "Bearer $env:GITHUB_TOKEN"
    Accept = "application/vnd.github+json"
    "X-GitHub-Api-Version" = "2022-11-28"
}

$BaseUrl = "https://api.github.com/repos/$Owner/$Repository"

function Get-GitHubPages {
    param([string]$Endpoint)

    $Results = @()
    $Page = 1

    do {
        $Url = "$BaseUrl/$Endpoint" + "?per_page=100&page=$Page"
        Write-Host "Requesting $Url"

        $Response = Invoke-RestMethod `
            -Uri $Url `
            -Headers $Headers `
            -Method Get

        $Results += @($Response)
        $Page++
    } while ($Response.Count -eq 100)

    return $Results
}

Write-Host "Retrieving Dependabot alerts..."
$Dependabot = Get-GitHubPages "dependabot/alerts"

Write-Host "Retrieving CodeQL alerts..."
$CodeScanning = Get-GitHubPages "code-scanning/alerts"

Write-Host "Retrieving Secret Scanning alerts..."
$SecretScanning = Get-GitHubPages "secret-scanning/alerts"

$Report = [PSCustomObject]@{
    Repository = "$Owner/$Repository"
    GeneratedAt = (Get-Date).ToUniversalTime().ToString("o")
    Summary = @{
        Dependabot = $Dependabot.Count
        CodeScanning = $CodeScanning.Count
        SecretScanning = $SecretScanning.Count
    }
    DependabotAlerts = $Dependabot
    CodeScanningAlerts = $CodeScanning
    SecretScanningAlerts = $SecretScanning
}

$OutputDirectory = Join-Path (Get-Location) "artifacts"

New-Item -ItemType Directory -Path $OutputDirectory -Force |
    Out-Null

$OutputFile = Join-Path $OutputDirectory "security-report.json"

$Report | ConvertTo-Json -Depth 30 |
    Set-Content -Path $OutputFile -Encoding utf8

Write-Host ""
Write-Host "Security report generated."
Write-Host "Repository: $Owner/$Repository"
Write-Host "Dependabot: $($Dependabot.Count)"
Write-Host "Code Scanning: $($CodeScanning.Count)"
Write-Host "Secret Scanning: $($SecretScanning.Count)"
Write-Host "Report: $OutputFile"
```

8.5 Execute the report

PowerShell – execute

```
$env:GITHUB_TOKEN = "YOUR_TOKEN"

.\scripts\security-report.ps1 `
    -Owner "YOUR-USERNAME" `
    -Repository "securebank-gh500"

Get-Content .\artifacts\security-report.json
```

> The sample script is a teaching baseline. Before production use, add endpoint-specific error handling, rate-limit/backoff behavior, robust pagination checks, redaction of sensitive fields and GitHub App authentication. Do not commit tokens or export them in an unsecured Postman collection.

8.6 Postman tests

Postman test – successful response

```
pm.test("GitHub API response successful", function () {
    pm.expect(pm.response.code).to.be.oneOf([200]);
});

pm.test("Response is JSON", function () {
    pm.response.to.be.json;
});
```

8.7 Advanced API topics

• REST API versus GraphQL API.

• Fine-grained PAT versus classic PAT.

• GitHub App versus user token.

• Repository and organization permission boundaries.

• Alert retrieval versus alert dismissal.

• Pagination, rate limits and conditional requests.

• Webhooks versus scheduled polling.

Stage 9 – GitHub Actions and secure CI/CD

Objective: combine functional tests, dependency review, CodeQL, Semgrep and branch governance into a secure pull request workflow.

9.1 Create CI workflow

Complete file: .github/workflows/ci.yml

```
name: SecureBank CI

on:
  push:
    branches:
      - main
  pull_request:
    branches:
      - main
  workflow_dispatch:

permissions:
  contents: read

jobs:
  test:
    name: Application Validation
    runs-on: ubuntu-latest
    steps:
      - name: Checkout
        uses: actions/checkout@v4

      - name: Setup Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.12"
          cache: pip

      - name: Install Dependencies
        run: |
          python -m pip install --upgrade pip
          pip install -r requirements-dev.txt

      - name: Run Unit Tests
        run: |
          python -m pytest tests -v

      - name: Validate Application Import
        run: |
          python -c "import app; print('Application imported successfully')"
```

9.2 Create a pull request

PowerShell – feature branch

```
git checkout -b feature/security-demo

# Make a small, safe application change.

git add .
git commit -m "Add SecureBank feature"
git push -u origin feature/security-demo
```

Open a pull request against main. Observe CI, Dependency Review, CodeQL and Semgrep checks. Secret Protection operates at the repository push boundary when configured; it should not be confused with a workflow job.

9.3 Configure branch rulesets

Setting	Suggested lab configuration
Target branches	main
Require pull request	Enabled
Required approvals	1 or 2
Require status checks	Enabled
Require code owner review	Enabled
Block force pushes	Enabled
Restrict deletion	Enabled
Require conversation resolution	Enabled

Add the relevant CI and security checks as required checks after they have completed successfully and GitHub has registered their check names. Verify exact check names in the repository.

9.4 Advanced GitHub Actions topics

Capability	Demonstration
Triggers	Push, pull_request, schedule, workflow_dispatch.
Job dependencies	needs and job ordering.
Conditional execution	if expressions.
Matrix builds	Multiple supported Python versions.
Artifacts	Retain scan reports and SBOMs.
Secrets	Secure workflow credentials.
Environments	Production approval gates.
Permissions	Least-privilege GITHUB_TOKEN.
Reusable workflows	Centralized security pipelines.
Concurrency	Avoid duplicate workflow execution.
Self-hosted runners	Enterprise runner security model.

9.5 Reusable workflow example

Illustrative caller job

```
jobs:
  enterprise-security:
    uses: YOUR-ORG/security-workflows/.github/workflows/python-security.yml@main
```

For production, pin reusable workflows and third-party actions to approved immutable commit SHAs. Govern workflow changes, runner groups, network access, artifact retention and token permissions.

9.6 Release gate exercise

Define a policy: critical findings block release; high findings require security approval; medium findings are tracked against an agreed SLA; low findings are monitored. A successful scanner run does not mean the application has no findings. Implement explicit policy enforcement and documented exceptions.

Stage 10 – Enterprise security governance

Objective: move from one repository to an enterprise operating model across many repositories.

10.1 Roles and responsibilities

Role	Responsibility
Enterprise administrators	Enterprise-wide policies and governance.
Organization administrators	Security configuration and access.
Security managers	Vulnerability monitoring and remediation oversight.
Repository administrators	Repository security configuration.
Developers	Secure development and remediation.
Platform engineering	Shared workflows and automation.
Auditors	Evidence and policy compliance.

10.2 Organization security configurations

• Review default security configurations.

• Assign configurations to appropriate repositories.

• Enable Code Security and Secret Protection where entitled.

• Set Dependabot policies and update behavior.

• Review security policy coverage and repository exceptions.

• Understand the Security Overview and repository-level visibility.

10.3 Security Overview

Demonstrate organization-level visibility into Dependabot alerts, code scanning alerts, secret scanning alerts, coverage, remediation progress, ownership and repository risk. Explain what an executive can learn without reviewing source code.

10.4 Security campaigns

Scenario: the security team discovers that 40 repositories use a vulnerable package. Identify affected repositories, prioritize critical applications, assign owners, coordinate remediation PRs, track progress, validate fixed versions and close the campaign with evidence.

10.5 Governance and exceptions

• Security Manager permissions and delegated responsibilities.

• Alert dismissal and delegated bypass governance.

• Ruleset enforcement and repository access.

• Custom security configurations and exceptions.

• Audit logs and evidence retention.

• Severity-based remediation SLAs.

> A security exception should include a business justification, named accountable owner, compensating controls, approval and an expiry date. Avoid permanent undocumented exceptions.

10.6 Enterprise security policy – final SECURITY.md

Complete file: SECURITY.md – enterprise version

```
# SecureBank Enterprise Security Policy
```

```
## 1. Purpose
This policy establishes security requirements for all SecureBank
software development repositories.

## 2. Security Controls
All production repositories must implement:
- Secret scanning and push protection.
- Dependency vulnerability monitoring.
- CodeQL or an approved code scanning alternative.
- Dependency review on pull requests.
- SBOM generation for production releases.
- Protected main branches.
- Mandatory code review.
- Least-privilege GitHub Actions permissions.

## 3. Vulnerability Classification
| Severity | Required Action |
|---|---|
| Critical | Immediate escalation and release block |
| High | Security approval and prioritized remediation |
| Medium | Remediation within agreed SLA |
| Low | Planned remediation and monitoring |

The organization must define actual remediation SLAs
based on regulatory and business requirements.

## 4. Vulnerability Management
Every security alert must have:
- An accountable owner.
- A severity classification.
- A remediation plan.
- A target resolution date.
- Closure evidence.

## 5. Secret Management
- Never commit production credentials.
- Enable push protection.
- Revoke exposed credentials.
- Rotate compromised credentials.
- Investigate unauthorized access.

## 6. Supply Chain Security
- Maintain dependency visibility.
- Enable Dependabot alerts.
- Review dependencies before merging.
- Generate SBOMs for releases.
- Review critical third-party vulnerabilities.

## 7. Code Security
- Run CodeQL or approved scanning.
- Maintain organization-specific queries.
- Review security findings.
- Prevent critical vulnerabilities from reaching production.

## 8. Governance
Security exceptions require:
- Business justification.
- Named approver.
- Compensating controls.
- Expiration date.
- Audit evidence.

## 9. Continuous Improvement
Security teams must periodically review:
- Alert trends.
- Remediation performance.
- Repository coverage.
- Security policy adoption.
- Scanner effectiveness.
- Supply chain risk.
```

Enterprise capstone – incident simulation

Scenario: a critical vulnerability is disclosed in a Python dependency used by SecureBank. The security team also discovers a potentially exposed credential and a critical CodeQL finding. A production release is scheduled.

Team mission

• Identify the vulnerable dependency using Dependabot.

• Identify affected components using the dependency graph and SBOM.

• Create and review a dependency remediation pull request.

• Investigate the secret scanning alert and demonstrate credential revocation procedures.

• Find and remediate the CodeQL vulnerability.

• Generate and inspect Semgrep SARIF results.

• Retrieve security alerts using the GitHub REST API.

• Generate a consolidated security report.

• Demonstrate that the release is blocked by the configured security policy.

• Present remediation evidence to the security manager.

Expected deliverables

• Working GitHub repository and application.

• Dependabot configuration and dependency review workflow.

• Secret Protection configuration evidence.

• CodeQL workflow and custom query.

• Semgrep workflow and SARIF evidence.

• SPDX and CycloneDX SBOMs.

• Postman collection for security APIs.

• Automated security report.

• CI/CD workflow and branch ruleset.

• Enterprise security policy and governance demonstration.

Final repository structure

Target structure after all stages

```
securebank-gh500/
■
■■■ app.py
■■■ requirements.txt
■■■ requirements-dev.txt
■■■ README.md
■■■ SECURITY.md
■■■ CODEOWNERS
■
■■■ tests/
■   ■■■ test_app.py
■
■■■ security-queries/
■   ■■■ avoid-eval.ql
■
■■■ scripts/
■   ■■■ generate-sbom.ps1
■   ■■■ security-report.ps1
■
■■■ artifacts/                # Generated outputs; normally not committed
■   ■■■ securebank-spdx.json
```

```
■   ■■■ securebank-cyclonedx.json
■   ■■■ security-report.json
■
■■■ .github/
    ■■■ dependabot.yml
    ■■■ workflows/
        ■■■ ci.yml
        ■■■ codeql.yml
        ■■■ dependency-review.yml
        ■■■ security.yml
```

Final README.md

Complete file: README.md

```
# SecureBank – GH-500 Enterprise Security Workshop

An end-to-end GitHub Advanced Security implementation.

## Scenario
SecureBank is a fictional banking application used to demonstrate
enterprise DevSecOps and software supply chain security.

## Prerequisites
- Git
- Python 3.12
- Visual Studio Code
- GitHub account
- PowerShell
- Postman
- Syft
- CodeQL CLI (optional for local analysis)

## Local Setup
```powershell
git clone https://github.com/YOUR-USERNAME/securebank-gh500.git
cd securebank-gh500
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -r requirements-dev.txt
```

Run Application
```powershell
python app.py
```
Application: http://127.0.0.1:5000

Run Tests
```powershell
python -m pytest tests -v
```

Security Capabilities
1. Dependabot
2. Dependency Graph
3. Dependency Review
4. Secret Scanning
5. Push Protection
6. CodeQL
7. Custom CodeQL Queries
8. SARIF
9. Semgrep
10. SBOM
11. GitHub REST API
12. GitHub Actions
13. Security Governance

Generate SBOM
```powershell
.\scripts\generate-sbom.ps1
```

Generate Security Report
```powershell
$env:GITHUB_TOKEN = "YOUR_TOKEN"
.\scripts\security-report.ps1 `
    -Owner "YOUR-USERNAME" `
    -Repository "securebank-gh500"
```

Important
The application deliberately contains vulnerable training code.
It must not be deployed to production. Complete the lab in an
isolated repository using synthetic data and credentials only.
```

Final validation commands

PowerShell – validation checklist

```
# Run functional tests
python -m pytest tests -v

# Review installed dependencies
python -m pip list

# Generate SBOM
.\scripts\generate-sbom.ps1

# Verify SBOM outputs
Test-Path .\artifacts\securebank-spdx.json
Test-Path .\artifacts\securebank-cyclonedx.json

# Check CodeQL CLI (if installed)
codeql version

# Check repository status and history
git status
git log --oneline -10
```

Recommended delivery plan

Session	Duration	Coverage
Day 1 – Session 1	1.5 hours	Stage 1: repository and threat modeling
Day 1 – Session 2	1.5 hours	Stage 2: Dependabot and supply chain
Day 1 – Session 3	1 hour	Stage 3: Secret Protection
Day 1 – Session 4	2 hours	Stages 4–5: CodeQL and custom queries
Day 2 – Session 1	1.5 hours	Stage 6: SARIF and third-party scanning
Day 2 – Session 2	1 hour	Stage 7: SBOM
Day 2 – Session 3	1.5 hours	Stage 8: REST APIs and Postman
Day 2 – Session 4	1.5 hours	Stage 9: GitHub Actions and CI/CD
Day 2 – Session 5	1 hour	Stage 10: enterprise governance
Final challenge	1 hour	Enterprise incident simulation

Production-readiness enhancements

Area	Production enhancement
GitHub Actions	Pin actions to immutable SHAs; review workflow changes.
Authentication	Use GitHub Apps and short-lived tokens.
Runner security	Use hardened, isolated runners and controlled network access.
CodeQL	Use organization-managed, tested query packs.
Dependency security	Define approved dependency and remediation policies.
Secret Protection	Govern delegated bypass and exceptions.
SBOM	Generate and retain an SBOM for every production release.
SARIF	Define scanner ownership, categories and deduplication.
API automation	Implement pagination, rate-limit handling and audit logging.
Governance	Use organization rulesets and security configurations.
Remediation	Define severity-based SLAs and security campaigns.
Compliance	Retain evidence and produce auditable reports.

Production-readiness warning: the sample app deliberately includes vulnerable code and outdated dependencies. It demonstrates detection and remediation, not a production banking implementation. Before production use, replace unsafe patterns, upgrade and validate dependencies, protect credentials, test the release and apply enterprise-approved controls.

Instructor notes and technical caveats

- Confirm GitHub plan, repository visibility and security entitlements before the workshop.
- Validate the current CodeQL Action versions and pin approved actions to immutable commit SHAs for production.
- The custom eval() query is intentionally simple; use a tested dataflow query as the advanced extension.

• The Semgrep auto configuration and SARIF upload must be validated against the current scanner and GitHub upload requirements.

• Syft's directory scan inventories files available in the scanned directory; compare its output with GitHub's dependency graph and release-specific SBOM process.

• The PowerShell API example is a starter lab; production automation needs robust error handling, rate-limit behavior, redaction and GitHub App authentication.

• Do not assume successful workflow execution means no vulnerabilities exist. Implement explicit policy checks and exception handling.

Official references

GitHub Advanced Security documentation: https://docs.github.com/en/code-security

Dependabot documentation: https://docs.github.com/en/code-security/dependabot

CodeQL documentation: https://codeql.github.com/docs/

GitHub Code Scanning: https://docs.github.com/en/code-security/code-scanning

GitHub Secret Protection: https://docs.github.com/en/code-security/secret-scanning

GitHub REST API: https://docs.github.com/en/rest

GitHub Actions security: https://docs.github.com/en/actions/security-for-github-actions

SARIF specification: https://docs.oasis-open.org/sarif/sarif/v2.1.0/sarif-v2.1.0.html

Syft documentation: https://github.com/anchore/syft

Semgrep documentation: https://semgrep.dev/docs/

GH-500 Enterprise Advanced Security Workshop

Appendix: Automating the GitHub Security Report with GitHub Actions

This appendix extends the workshop's PowerShell security-reporting exercise. It moves the report from a local/manual run into a scheduled and manually triggered GitHub Actions workflow. The workflow calls the GitHub REST API through the existing **scripts/security-report.ps1** script and stores the generated JSON report as a downloadable workflow artifact.

1. What the automation does

• Runs every day at 03:30 UTC (09:00 India Standard Time).

• Can also be started manually from the Actions tab using **workflow_dispatch**.

• Runs on a Windows-hosted runner and executes the script with PowerShell 7 (**pwsh**).

• Reads Dependabot, code-scanning, and secret-scanning alerts through GitHub REST API endpoints.

• Uploads timestamped JSON output as an Actions artifact retained for 30 days.

2. Required repository structure

```
securebank-gh500/
■■■ scripts/
■   ■■■ security-report.ps1
■■■ .github/
    ■■■ workflows/
        ■■■ security-report.yml
```

Keep the PowerShell script from the preceding workshop stage in **scripts/security-report.ps1**. It should accept the parameters **-Owner**, **-Repository**, and **-OutputDirectory**, read its token from the **GITHUB_TOKEN** environment variable, and write a file matching **security-report-*.json** into the output directory.

3. Complete workflow file

```
name: SecureBank Security Report

on:
  schedule:
    - cron: "30 3 * * *"
  workflow_dispatch:
  push:
    branches: [main]
    paths:
      - "scripts/security-report.ps1"
      - ".github/workflows/security-report.yml"

permissions:
  contents: read

jobs:
  security-report:
    runs-on: windows-latest

    steps:
      - name: Check out repository
        uses: actions/checkout@v4

      - name: Check PowerShell
        shell: pwsh
        run: $PSVersionTable

      - name: Generate security report
        shell: pwsh
        env:
          GITHUB_TOKEN: ${{ secrets.SECURITY_REPORT_TOKEN }}
        run: |
          ./scripts/security-report.ps1 `
            -Owner "${{ github.repository_owner }}" `
            -Repository "${{ vars.SECUREBANK_REPOSITORY }}" `
            -OutputDirectory "./artifacts"

      - name: Upload security report
        uses: actions/upload-artifact@v4
```

```
          with:
            name: securebank-security-report
            path: artifacts/security-report-*.json
            retention-days: 30
            if-no-files-found: error
```

Configuration note: define the repository Actions variable **SECUREBANK_REPOSITORY** with the repository name (for example, **securebank-gh500**). This works consistently for scheduled, manual, and push-triggered runs. The owner is supplied automatically from **github.repository_owner**.

4. Configure the token securely

1. In GitHub, open the repository → **Settings** → **Secrets and variables** → **Actions**.

2. Under **Repository secrets**, create **SECURITY_REPORT_TOKEN** and paste the fine-grained token value.

3. Under **Repository variables**, create **SECUREBANK_REPOSITORY** and set it to the repository name.

4. Limit the fine-grained token to the target repository. Grant only the read permissions required by the APIs used in the script.

Fine-grained token permission	Access
Metadata	Read-only
Dependabot alerts	Read-only
Code scanning alerts	Read-only
Secret scanning alerts	Read-only

Never commit the token to the repository, put it in the workflow YAML, print it in logs, or include it in the JSON report. Treat secret-scanning alert metadata and downloaded reports as sensitive security information.

5. Run and verify the workflow

• Commit the workflow file to the default branch at **.github/workflows/security-report.yml**.

• Open the repository's **Actions** tab and select **SecureBank Security Report**.

• Select **Run workflow** to start a manual execution.

• Open the run and inspect each step. Confirm that the report-generation step completes successfully.

• Open the run's **Artifacts** section and download **securebank-security-report**.

• Open the JSON file locally and validate the repository name, generation timestamp, and alert summary counts.

6. Operational notes and troubleshooting

403 / 404 from an alert endpoint: Check that the token is scoped to the correct repository and has the corresponding read permission. Some alert APIs also depend on repository visibility, security-feature availability, and organization policy.

GITHUB_TOKEN versus fine-grained PAT: The built-in GITHUB_TOKEN may be sufficient in some repository configurations when the required workflow permissions are available. If the alert endpoints reject it, use the narrowly scoped fine-grained token stored as SECURITY_REPORT_TOKEN or, for enterprise production, a GitHub App installation token.

No artifact found: Check that the script created a file under ./artifacts and that its name matches security-report-*.json. The workflow intentionally fails if no matching file is produced.

Scheduled run did not start at the exact minute: GitHub scheduled workflows can be delayed during periods of high load. They run from the default branch, and schedules may be disabled for inactive public repositories.

PowerShell header or pagination errors: Validate the response-header handling in security-report.ps1 on the runner's PowerShell version. Follow GitHub's Link header for pagination and handle rate limits or endpoint-specific 403/404 responses deliberately.

7. Enterprise hardening recommendations

• Prefer a GitHub App installation token over a long-lived personal access token for organization-wide reporting.

• Pin third-party actions to reviewed full commit SHAs in production rather than relying only on mutable version tags.

• Use least-privilege permissions and keep the workflow's explicit **permissions** block.

• Restrict who can read workflow artifacts; reduce retention if the report contains sensitive alert metadata.

• Add controlled retries for HTTP 429 and transient 5xx responses, and record endpoint-level failures without exposing token values or secret contents.

• For multiple repositories, iterate over an approved repository inventory and create one report per repository or a consolidated enterprise report.

8. Demonstration checklist

■ Manual run succeeds from the Actions tab.

■ Scheduled trigger is present and uses UTC time correctly (03:30 UTC = 09:00 IST).

■ The workflow can read all three alert categories where enabled and permitted.

■ The JSON artifact is downloadable and contains the expected summary and alert collections.

■ The token is stored only as a GitHub Actions secret and is never printed.

■ A deliberately failing token/permission test produces a clear, safe diagnostic.

Important accuracy note

The workflow automates collection and packaging of alert data; it does not remediate vulnerabilities or guarantee that every alert endpoint is enabled for every repository. Validate the exact API permissions and repository security settings in the target GitHub organization before using this as an enterprise reporting control.